

Entrega 1 · El tablero con piezas

Contexto

Primera entrega evaluable del proyecto de ajedrez — la primera que se entrega y se califica de verdad. En el Bloque 1 se hizo, como práctica de clase (sin nota), un ejercicio de representar un tablero 8×8 alternando dos caracteres; ahora el tablero debe contener información real: qué pieza hay en cada casilla, no solo si es clara u oscura.

Lo que ya dominas al llegar aquí

Bloque 1 — variables anotadas, tipos, operadores, entrada/salida, casting, comentarios.

Bloque 2 — condicionales (incluidos anidados), bucles for/while, break/continue, bucles anidados, y la idea de máquina de estados.

Bloque 3 — listas, tuplas y diccionarios: la herramienta que te falta para guardar el tablero completo.

Objetivo

Representar el tablero de ajedrez completo, con las 32 piezas en su posición inicial estándar, usando una estructura de datos (lista de listas o diccionario) — y permitir consultar el contenido de cualquier casilla.

Unicode: cómo escribir estos símbolos

Hasta ahora tus programas solo han usado caracteres del teclado (letras, dígitos, algunos símbolos). Unicode es el estándar que usa Python — y prácticamente todo el software actual — para representar CUALQUIER carácter existente: alfabetos de todos los idiomas, símbolos matemáticos y, entre otras muchas cosas, las 12 piezas de ajedrez. Cada carácter Unicode tiene asignado un número único, su "punto de código", que se escribe en hexadecimal precedido de "U+" (p. ej. U+2654).

En tu código puedes escribir cada símbolo de dos formas EQUIVALENTES — el resultado es exactamente el mismo carácter:

- Escribiéndolo tal cual, copiando el símbolo directamente desde la tabla de abajo (por ejemplo, para el rey blanco: rey_blanco: str seguido del símbolo pegado entre comillas).
- Con su secuencia de escape \uXXXX (XXXX es el punto de código en hexadecimal, sin el "U+"): rey_blanco: str = "\u2654"

La segunda forma es más fiable si tu teclado no tiene esos símbolos o si copiar y pegar te da problemas de codificación: puedes escribir \u2654 con las teclas normales de tu teclado y Python lo convierte automáticamente en el símbolo correspondiente al ejecutar el programa.

Pieza	Color	Símbolo	Punto de código	En Python (alternativa)
Rey	Blanca		U+2654	"\u2654"
Dama	Blanca		U+2655	"\u2655"
Torre	Blanca		U+2656	"\u2656"
Alfil	Blanca		U+2657	"\u2657"

Pieza	Color	Símbolo	Punto de código	En Python (alternativa)
Caballo	Blanca		U+2658	"\u2658"
Peón	Blanca		U+2659	"\u2659"
Rey	Negra		U+265A	"\u265A"
Dama	Negra		U+265B	"\u265B"
Torre	Negra		U+265C	"\u265C"
Alfil	Negra		U+265D	"\u265D"
Caballo	Negra		U+265E	"\u265E"
Peón	Negra		U+265F	"\u265F"

Antes de nada, comprueba que estos 12 símbolos se ven bien en tu terminal (copia la tabla o escribe `print("\u2654")` y ejecuta). La terminal integrada de VS Code los muestra correctamente. Si en la tuya no se ven, coméntalo con el profesorado antes de avanzar.

La posición inicial exacta, casilla por casilla

Para que no haya ninguna duda sobre dónde va cada pieza, aquí tienes el tablero completo tal y como debe quedar nada más arrancar el programa ("." indica casilla vacía):

	a	b	c	d	e	f	g	h
8								
7								
6	.	.	.	.	.	.	.	.
5	.	.	.	.	.	.	.	.
4	.	.	.	.	.	.	.	.
3	.	.	.	.	.	.	.	.
2								
1								

Requisitos funcionales

1. El tablero se guarda en una única estructura de datos (lista de listas de 8x8, o un diccionario indexado por (fila, columna) o por la notación "e4"). Nada de 64 variables sueltas ni de una única cadena larga.
2. Cada pieza se representa con su símbolo Unicode de ajedrez real:       para las piezas blancas y       para las negras — el propio símbolo ya distingue el color, así que no hace falta mayúscula/minúscula. Las casillas vacías, con un carácter neutro (p. ej. ".").

3. Comprueba que los símbolos se ven correctamente en tu terminal antes de dar el ejercicio por terminado (la terminal integrada de VS Code los muestra bien). Si tu terminal no los soporta, coméntalo con el profesorado.
4. Al iniciar el programa, la estructura de datos se rellena con la posición inicial estándar de una partida de ajedrez — exactamente la que se muestra en la tabla de este documento, casilla por casilla.
5. El tablero completo se imprime por consola con coordenadas (columnas a-h, filas 1-8), recorriendo la estructura de datos con bucles anidados — no con `print()` repetidos.
6. El programa permite introducir una casilla por teclado (p. ej. "e2") e informa, con un mensaje que incluya el NOMBRE de la pieza (por ejemplo "Hay un caballo negro en b8"), no solo su símbolo, de qué pieza hay en ella, o de que está vacía.
7. El bucle principal del programa debe ser una máquina de estados, tal y como se explicó en el Bloque 2 — no un `while` con una simple bandera booleana (por ejemplo, `continuar = True/False` no es válido si no va acompañado de la estructura siguiente). Ver el esqueleto obligatorio más abajo.
8. Declara una variable de estado, llamada exactamente `estado` (str), con los valores "turno_blancas" / "turno_negras" (y un tercer valor para salir del bucle). El estado inicial es SIEMPRE "turno_blancas": la partida (simulada) empieza obligatoriamente con las blancas.
9. Los turnos deben respetarse estrictamente: se alternan en el orden correcto (blancas, negras, blancas, negras...) tras cada consulta a una casilla con formato válido — tenga o no pieza —, sin saltarse ninguno ni repetirse. Todavía no hay movimientos reales: es una simulación del turno. El comportamiento exacto se detalla más abajo.
10. Si la casilla introducida no tiene un formato válido (una letra de la a a la h seguida de un dígito del 1 al 8), el programa lo detecta con condicionales y muestra "Casilla no válida", sin interrumpirse y SIN cambiar el turno.
11. El programa termina cuando el usuario escribe exactamente la palabra "salir" (en minúsculas, sin comillas ni espacios) en el mismo sitio donde se pide una casilla.
12. Todas las variables declaradas, inicializadas y anotadas con su tipo, siguiendo la misma regla del Bloque 1.

Cómo debe comportarse el programa, paso a paso

Esta sección no añade requisitos nuevos: describe con precisión cómo se comportan los de arriba, para que no haya que adivinar nada.

1. Al arrancar, el programa crea la estructura de datos con la posición inicial completa (ver tabla anterior) y una variable `estado`: `str = "turno_blancas"`.
2. Inmediatamente después, imprime el tablero completo una vez, con sus coordenadas.
3. A continuación, entra en un bucle que se repite hasta que el usuario escriba "salir":
4. Dentro del bucle, primero se muestra un mensaje indicando de quién es el turno actual (por ejemplo: "Turno de las blancas").
5. Después, se pide una casilla por teclado (por ejemplo: "Casilla a consultar (o 'salir' para terminar): ").
6. Si el usuario ha escrito "salir", el programa termina inmediatamente, sin mensajes adicionales.
7. Si lo escrito no tiene formato de casilla válida, se muestra "Casilla no válida" y se vuelve al punto 4 (se pide de nuevo el turno y la casilla) SIN cambiar el estado.
8. Si la casilla es válida, se muestra el nombre de la pieza que contiene (o que está vacía) y el estado cambia (de `turno_blancas` a `turno_negras`, o viceversa). El bucle vuelve al punto 4.

Esqueleto obligatorio del bucle principal

Tu bucle principal debe seguir exactamente esta forma (los comentarios indican qué va en cada hueco; los nombres de las variables son libres salvo estado, que debe llamarse así):

```
estado: str = "turno_blancas"    # SIEMPRE empieza así

while estado != "salir":

    if estado == "turno_blancas":
        # mostrar 'Turno de las blancas'
        # pedir la casilla
        # si el usuario escribe "salir" -> estado = "salir"
        # si el formato no es válido -> mostrar aviso, estado NO cambia
        # si es válida -> mostrar qué hay, estado = "turno_negras"

    elif estado == "turno_negras":
        # exactamente el mismo procedimiento, y al terminar:
        # estado = "turno_blancas"
```

No se admite sustituir esto por un while con una bandera booleana (p. ej. jugando = True / jugando = False) sin la estructura if/elif sobre estado: la máquina de estados vista en el Bloque 2 es un requisito, no una sugerencia. Los comentarios de dentro de cada if los debes reemplazar por tu código real.

Ejemplo de ejecución

El tablero se imprime una sola vez, al principio, exactamente como en la tabla de posición inicial de más arriba. Después, así debería verse la interacción por teclado:

```
Turno de las blancas
Casilla a consultar (o 'salir' para terminar): b8
Hay un caballo negro en b8.

Turno de las negras
Casilla a consultar (o 'salir' para terminar): e4
La casilla e4 está vacía.

Turno de las blancas
Casilla a consultar (o 'salir' para terminar): z9
Casilla no válida.

Turno de las blancas
Casilla a consultar (o 'salir' para terminar): salir
```

Fíjate en el orden: tras consultar b8 (válida, con pieza) el turno pasa a las negras; tras consultar e4 (válida, vacía) vuelve a las blancas; tras z9 (formato inválido) el turno NO cambia — se repite "Turno de las blancas". Los textos exactos de tus mensajes pueden variar, pero el comportamiento (cuándo cambia el turno y cuándo no) debe ser este.

Lo que todavía no se pide

- Funciones propias: todo el código vive en el cuerpo principal del script (las funciones llegan en el Bloque 4).
- Clases ni programación orientada a objetos (Bloque 6).
- Movimientos reales de las piezas ni comprobación de si un movimiento es válido (Entrega 2).
- Manejo de excepciones con try/except (Bloque 7): los errores de entrada se gestionan con condicionales.

Formato y plazo de entrega

- Crea un repositorio git para el proyecto (o continúa el que ya tuvieras del Bloque 1): un commit por avance significativo, con mensajes descriptivos.
- Un único archivo .py, ejecutable con `python nombre_archivo.py`, sin errores.
- Entrega a través del campus virtual dentro del plazo indicado tras el Bloque 3.

Criterios de evaluación

Además de los criterios generales de la parte práctica (funcionalidad, claridad, ausencia de warnings, no duplicidad, aplicación correcta de los conceptos del bloque), en esta entrega se valorará especialmente:

- La estructura de datos elegida es adecuada y se usa de forma consistente en todo el programa.
- El tablero impreso es visualmente correcto y coincide EXACTAMENTE con la tabla de posición inicial de este documento.
- Las consultas de casilla funcionan para cualquier entrada válida, y gestionan con elegancia las inválidas, con el comportamiento de cambio de turno descrito arriba.

Antes de entregar, comprueba

- He ejecutado el programa desde cero y el tablero impreso coincide, casilla por casilla, con la tabla de posición inicial de este documento.
- Mi bucle principal sigue el esqueleto obligatorio: `while + if/elif` sobre estado — no una bandera booleana simple.
- La partida empieza siempre con "turno_blancas", y los turnos se alternan en el orden correcto (blancas, negras, blancas...) sin saltarse ninguno.
- He probado, como mínimo: una casilla con pieza blanca, una con pieza negra, una vacía, una con formato inválido, y escribir "salir".
- Tras cada consulta válida el turno mostrado cambia; tras una inválida, no cambia — lo he comprobado a propósito.
- No hay ninguna variable sin su tipo anotado (salvo las de los bucles `for`, que nunca se anotan).
- `git log` muestra varios commits con mensajes descriptivos, no uno único al final.
- El archivo se ejecuta con `python nombre_archivo.py` sin errores, y VS Code no marca ningún warning.

Esta entrega es la base sobre la que se construirá la Entrega 2: cuanto más ordenada esté la estructura de datos ahora, menos trabajo de refactorización tendrás más adelante.